

Technical README & Architectural Overview

PROJECT OBJECTIVE & PHILOSOPHY

The primary purpose of the Monnit Utility is to provide a robust, local-first interface for direct integration with the MODBUS TCP interface of the Monnit ALTA Ethernet Gateway 4 (firmware version 1.1.0.6 or later). While its core function is to unlock the potential of Monnit's industrial hardware with guidance to avoid common configuration pitfalls without reliance on cloud services, the application significantly expands upon this by serving as a bidirectional data hub. It is designed not only to poll and visualize Monnit sensor data but also to broadcast this data to external systems (via MQTT, BACnet, Webhooks) and receive data from third-party sources (via SNMP, LoRaWAN, MQTT), unifying diverse industrial protocols into a single, coherent dashboard.

CORE GOALS:

1. **Guided Management:** The application aims to lower the barrier to entry for industrial hardware configuration. It maintains a "single pane of glass" for Monnit Gateway configuration via an intelligent HTTP Proxy and a guided setup wizard. This wizard actively prevents common pitfalls, such as IP conflicts or locking configuration errors, by validating settings in real-time before applying them to the hardware.
2. **Local Reliability (The "Local-First" Approach):** In an era of cloud dependency, this utility champions a "local-first" architecture. It provides a serverless solution using an embedded SQLite database, ensuring that core monitoring, data persistence, and alerting functionality remain fully operational even during total internet outages. This autonomy is critical for high-security environments, air-gapped networks, or remote facilities with unreliable connectivity, ensuring that data ownership and operational continuity remain with the user.
3. **Extensible Modularity:** To ensure the application can adapt to future needs without bloating the core codebase, it leverages a robust Plugin Manager architecture. This design allows for rapid expansion by integrating new data sources ("Receive" plugins) and data distribution methods ("Broadcast" plugins). Developers can add support for entirely new protocols or integrations by writing isolated modules, ensuring the core stability of the application is never compromised by peripheral features.



Monnit MODBUS TCP Utility

4. **Protocol Consolidation:** The primary objective is to unify multiple, often incompatible network protocols into a cohesive data stream. This involves bridging the gap between proprietary hardware (Monnit MODBUS) and widely adopted open standards (MQTT, BACnet, SNMP, LoRaWAN). Instead of requiring separate tools for each device type, this utility acts as a "Universal Translator," normalizing incoming data into a standardized internal format that can be visualized on a single, easy-to-use dashboard.

SYSTEM ARCHITECTURE: The Local Data Hub

The application employs a monolithic client-server architecture. This design choice simplifies deployment and minimizes resource overhead, allowing the app to act as a trusted local intermediary between the user's application, the industrial devices, and other network systems.

1. Backend Server standalone application: The Central Nervous System
 - **Platform:** The backend runs as a native application.
 - **Data Persistence:** Storage is handled by an embedded SQLite database (sensor_data.db) utilizing the better-sqlite3 library. This library was chosen specifically for its ability to perform high-performance synchronous operations, which simplifies the logic flow and prevents the race conditions often associated with asynchronous database writes in high-throughput logging scenarios. The number of stored records is user-configurable.
 - **Data Acquisition (Polling):** The core engine manages the MODBUS TCP Polling Engine for Monnit Gateways (Port 502). To ensure stability on the resource-constrained embedded hardware of the gateways, the poller implements controlled pacing (a 1-second inter-batch delay). This "gentle polling" strategy prevents socket exhaustion and CPU spikes on the gateway, ensuring long-term reliability.
 - **Data Ingestion (Receiving):** The backend orchestrates all "Receive" Plugins via the backend Plugin Manager. It manages the lifecycle of listeners for external protocols, ensuring ports are bound correctly and errors are handled gracefully.
 - *MODBUS Poller:* Actively connects to external third-party MODBUS devices to read specific registers.
 - *SNMP Trap Listener:* Opens a UDP socket to passively listen for asynchronous SNMP alerts from network infrastructure.



Monnit MODBUS TCP Utility

- *MQTT Subscriber*: Maintains a persistent connection to an MQTT broker to ingest JSON payloads.
 - *BACnet Subscriber*: Discovers BACnet devices via 'Whols' broadcasts and logs their presence.
 - *LoRaWAN Listener*: Decrypts and ingests raw uplink data from Semtech UDP Packet Forwarders (Port 1700), effectively turning the utility into a lightweight LoRaWAN Network Server.
 - **Data Distribution (Broadcasting)**: To facilitate integration with other systems, the backend directs normalized Monnit and third-party data to external systems via Broadcast Plugins (Webhooks, MQTT, BACnet/IP).
 - **Gateway Proxy**: A sophisticated proxy layer fetches, parses, and sanitizes the Monnit Gateway's native HTML configuration pages. This allows users to configure advanced gateway settings directly within the utility's modern UI, allowing the user to configure the gateway directly without added complexity of a separate browser interface.
2. Frontend Client: The Presentation Layer
- **Technology**: The frontend is built as a vanilla HTML, CSS, and JavaScript Single Page Application (SPA). This dependency-free approach ensures maximum speed and minimal build complexity.
 - **Communication Bridge**:
 - *Application Mode*: Uses a secure IPC (Inter-Process Communication) Bridge defined in preload.js. This bridge mimics the Socket.IO interface, allowing the exact same frontend code to run in both web and desktop environments without modification.
 - **Visualization**: The UI renders dynamic device data cards, supports custom naming and color-coding for organizing complex sensor arrays, and provides comprehensive historical charting using chart.js.
 - **Dashboard Views**: The interface is strictly separated into "Monnit Devices" and "External Devices" tabs. This logical separation helps users distinguish between deeply integrated native sensors and third-party data streams.

3. Application Lifecycle & Updates

To ensure long-term stability and security without requiring manual user intervention, the application incorporates a self-update mechanism powered by electron-updater.

- **Source of Truth:** Updates are fetched securely from the official GitHub Repository Releases.
- **Automatic Mode:** A background scheduler checks for new versions once every 7 days. If a new version (e.g., v2.0.5) is detected, it is downloaded silently in the background.
- **Manual Mode:** Administrators can trigger an immediate check via the Settings interface.
- **Safety:** The update process verifies the integrity of the downloaded package before applying it. The application uses a "Restart to Install" model, ensuring that active polling sessions are only interrupted when the user is ready.

DATA STORAGE LOCATION

When running as an installed application, the application adheres to strict OS platform guidelines. User data is NOT stored in the program installation folder (generally, Program Files), which is often read-only. Instead, it is stored in the user's AppData directory to ensure data persists across application updates and system reboots.

Default Path: C:\Users\[User]\AppData\Roaming\monnit-modbus-tcp-utility\

Files located within this directory:

- **sensor_data.db:** The critical SQLite database file containing the complete history of all sensor readings, user settings, gateway configurations, and plugin states. Backing up this single file backs up the entire application state.
- **setup.lock:** A simple lock file that acts as a sentinel flag. Its presence indicates that the initial setup wizard has been successfully completed, preventing the application from forcing the user through the setup process on subsequent launches.
- **Logs:** Depending on the logging configuration, application debug logs and error reports may also be written to this directory for troubleshooting purposes.

Troubleshooting Note: If the application fails to load due to a corrupted state, or if you wish to perform a "hard reset" (e.g., to re-run the setup wizard from scratch), you can safely

Monnit MODBUS TCP Utility

delete the contents of this folder. Upon the next launch, the application will detect the missing files and automatically regenerate a fresh, empty database and a new lock file.

DATABASE SCHEMA OVERVIEW

The database schema is designed as a hybrid model, blending structured, specific tables for Monnit hardware with flexible, generalized tables for external device integration. This allows the system to be rigid where necessary (for specific Monnit features) and flexible where needed (for unknown third-party payloads).

Key Tables:

Table Name	Purpose & Key Fields
sensor_data	The primary historical log for Monnit sensor readings. It stores normalized values (voltage, rssi) alongside the raw data blob (rawData) and the alert state (alertTriggered).
gateway_data	Stores the inventory of known Monnit Gateways. It tracks network status (ip, macAddress) and critical configuration flags (isUnlocked, isReadOnly, isModbusActive) used by the setup wizard logic.
external_devices	The master table for all non-Monnit devices (MQTT, SNMP, Modbus, BACnet). It uses a text-based primary key (id) and stores the most recent payload as a JSON string (lastData), allowing it to accommodate any data structure.
external_device_history	A simplified historical log for external device payloads. It links to external_devices via foreign key and stores the raw JSON payload for every event.
sensor_metadata	Stores user-customizations for Monnit sensors, such as customName and colorGroup. Separating this from sensor_data prevents data duplication.
external_device_metadata	Equivalent to sensor_metadata but for external devices. It ensures that user renaming persists even if the external device sends a new payload.
alert_configs	Contains the user-defined threshold logic for the alerting engine (e.g., "Alert if Sensor 12345 is above 80 degrees").



Monnit MODBUS TCP Utility

settings

A global key-value store for application-wide preferences, such as email server configuration, port numbers, and plugin enable/disable states.

Direct Database Interaction (Advanced)

For advanced users, integrators, or IT administrators who wish to query the data directly without using the UI, the underlying SQLite database is accessible.

Warning: Direct interaction with the database should be read-only to prevent data corruption. Ensure the application is closed before performing any write operations or backups.

Database File Location (Windows)

The SQLite database file is located in the user's AppData directory:

C:\Users\[USERNAME]\AppData\Roaming\monnit-modbus-tcp-utility\sensor_data.db

You can open this file with any standard SQLite viewer (e.g., DB Browser for SQLite, DBeaver) or query it programmatically using Python, Node.js, PowerShell, etc.

Database Schema Overview

The database consists of several key tables:

- **sensor_data:** The primary table storing historical sensor readings.
- **gateway_data:** Stores configuration and status for known Monnit Gateways.
- **sensor_metadata:** Stores user-defined names and color groups for sensors.
- **alert_configs:** Stores user-defined alert thresholds.
- **external_devices:** Stores the latest state of non-Monnit devices (MQTT, SNMP, etc.).

Table: sensor_data

Column	Type	Description
id	INTEGER	Primary Key (Auto-incrementing)
sensorId	INTEGER	The unique ID of the sensor
gatewayId	INTEGER	The unique ID of the gateway that collected the data
deviceType	INTEGER	The Monnit sensor type code (e.g., 2 for Temp)



Monnit MODBUS TCP Utility

voltage	REAL	Battery voltage of the sensor
rss	INTEGER	Signal strength percentage
isAware	INTEGER	1 if sensor is in "Aware" state, 0 otherwise
rawData	TEXT	JSON string containing the full payload (useful for complex types)
timestamp	INTEGER	Unix timestamp (seconds) of the reading
alertTriggered	INTEGER	1 if this reading triggered an alert, 0 otherwise

Example SQL Queries

Here are common SQL queries you might use to extract data for reporting or integration.

1. Get the last 10 readings for a specific sensor: Replace 12345 with your actual Sensor ID.

```
SELECT
    datetime(timestamp, 'unixepoch', 'localtime') as time,
    sensorId,
    voltage,
    rss,
    rawData
FROM sensor_data
WHERE sensorId = 12345
ORDER BY timestamp DESC
LIMIT 10;
```

2. Get the average daily voltage for all sensors (Battery Health Check):

```
SELECT
    date(timestamp, 'unixepoch', 'localtime') as day,
    sensorId,
    AVG(voltage) as avg_voltage
```

```
FROM sensor_data
```

```
GROUP BY day, sensorId
```

```
ORDER BY day DESC;
```

3. Find all sensors that have triggered an alert in the last 24 hours:

```
SELECT
```

```
    datetime(timestamp, 'unixepoch', 'localtime') as alert_time,
```

```
    sensorId,
```

```
    rawData
```

```
FROM sensor_data
```

```
WHERE alertTriggered = 1
```

```
    AND timestamp > (strftime('%s', 'now') - 86400)
```

```
ORDER BY timestamp DESC;
```

4. Join Sensor Data with Custom Names: This query retrieves data but replaces the numeric ID with the custom name you assigned in the UI.

```
SELECT
```

```
    datetime(d.timestamp, 'unixepoch', 'localtime') as time,
```

```
    COALESCE(m.customName, 'Sensor ' || d.sensorId) as sensor_name,
```

```
    d.voltage,
```

```
    d.rssi
```

```
FROM sensor_data d
```

```
LEFT JOIN sensor_metadata m ON d.sensorId = m.sensorId
```

```
ORDER BY d.timestamp DESC
```

```
LIMIT 50;
```

5. Get the latest reading for every sensor (Current State):

```
SELECT
```

```
    d.sensorId,
```




Monnit MODBUS TCP Utility

```
datetime(MAX(d.timestamp), 'unixepoch', 'localtime') as last_seen,  
d.voltage,  
d.rssi,  
d.isAware  
FROM sensor_data d  
GROUP BY d.sensorId;
```

CORE LOGIC: THE PLUGIN MANAGER

The Plugin Manager serves as the traffic controller for the entire application. It abstracts the complexity of individual modules, ensuring a single, unified entry point for all data flowing into or out of the utility. It enforces a standard lifecycle for all plugins.

Key Plugin Functions:

Function	Triggered By	Plugin Action
.init(deps)	startup	The manager loads all plugin modules and injects the core dependencies (db wrapper, log function, io instance). This dependency injection allows plugins to interact with the database and frontend without needing to require core files directly.
.broadcastData(sensors, gw)	Main MODBUS Polling Cycle success	Immediately after a successful poll, this function routes the fresh sensor data to all enabled <i>Broadcast</i> Plugins (Webhooks, MQTT, BACnet). This ensures external systems are updated in near real-time.
.broadcastAlert(message, reading)	Alert Engine detects threshold breach	When the internal alerting engine flags a violation, this function routes the specific alert details to all enabled <i>Broadcast</i> Plugins, allowing for redundant notification paths (e.g., Email + MQTT).



Monnit MODBUS TCP Utility

plugin.receive(data)

(Internal to
Receive
Plugins)

Logic strictly contained within *Receive* plugins (like snmptrap.js or modbus_poller.js). It processes raw incoming data (UDP packets, TCP streams) and standardizes it before saving it to the external_devices tables.